

CPE393: Introduction to Data Science with Python
Summer Semester 2026 – King Mongkut's University of Technology Thonburi (KMUTT)
Computer Engineering Department

Predicting Airbnb Listing Prices in Major US Cities

Group 15 K/B

Mathieu

Serghei

Samy

Exchange students from ESIEA

Instructors: Dr. Aye Hninn Khine and Mr. Naveed Sultan

Project type: Regression
Target variable: log_price
Dataset: AirBnB Listings in Major US Cities (Deloitte ML, Kaggle)
GitHub: <https://github.com/Lolilop668/Final-Project.git>

Submitted July 6, 2026

Contents

1	Project title and members	3
2	Problem definition and goals	3
2.1	Problem statement	3
2.2	Target variable	3
2.3	Why the problem matters	3
2.4	Expected outcome	3
3	Dataset description	4
3.1	Source and format	4
3.2	Cities covered	4
3.3	Features	4
3.4	Target and target range	4
3.5	Limitations of the data	4
4	Preprocessing and cleaning	5
4.1	Train and test split done by us	5
4.2	Dropping unused columns	5
4.3	Missing values	5
4.4	Encoding categorical variables	5
4.5	Scaling	5
5	Data visualization and exploratory data analysis	6
5.1	Distribution of the numeric features	6
5.2	Distribution of the categorical features	6
5.3	The target variable	7
5.4	Price by city and by room type	8
5.5	Correlation with the target	8
6	Feature engineering	9
6.1	Feature selection	9
6.2	Encoding and scaling as engineered features	9
6.3	Latitude and longitude as location proxies	10
6.4	What we chose not to do	10
7	Machine learning model training	10
7.1	Models	10

7.2	Training setup	10
7.3	Why these models	10
8	Evaluation metrics and results	11
8.1	Metrics	11
8.2	Validation results	11
8.3	Feature importance	11
8.4	Final results on the held-out test set	12
8.5	Honest reading of the results	13
9	Responsible AI practices	14
9.1	Privacy and personal data	14
9.2	Bias and fairness	14
9.3	Data leakage	14
9.4	Honest reporting	14
9.5	Real-world risks	14
9.6	Use of AI tools	14
10	Conclusion and future work	15
10.1	Main findings	15
10.2	What we learned	15
10.3	Future work	15
11	Team contribution	15
	References	16
	Acknowledgements	16

1 Project title and members

Project title: Predicting Airbnb Listing Prices in Major US Cities.

Group: 15 K/B.

Members: Mathieu, Serghei, and Samy. We are three exchange students from EISEA taking CPE393 at KMUTT during the summer 2026 semester.

GitHub repository: https://github.com/<your-username>/CPE_Project_Group15

Dataset: AirBnB Listings in Major US Cities, published for a Deloitte Machine Learning competition and hosted on Kaggle: <https://www.kaggle.com/datasets/rudymizrahi/airbnb-listings-in-major-us>

This report describes the full workflow we followed, from cleaning the raw data to training and testing three regression models. Every figure in the report comes directly from our notebook, and the numbers we quote match the outputs of the code we submitted.

2 Problem definition and goals

Setting the right nightly price on Airbnb is not easy. Hosts want a number that fills their calendar without leaving money on the table, and guests want to know whether a listing is a fair deal or not. Airbnb itself offers a price suggestion tool, but the logic behind it is hidden. We wanted to see how far we could get with public features alone and a few standard machine learning models.

2.1 Problem statement

Given the characteristics of an Airbnb listing, such as how many guests it fits, how many bedrooms and bathrooms it has, what type of room it is, and where it is located, can we predict its price? This is a supervised regression task. The value we predict is a continuous number, not a category.

2.2 Target variable

The target is `log_price`, the natural logarithm of the nightly price in US dollars. The dataset already stores the price in log form. Raw prices are heavily skewed to the right, with a small number of very expensive listings pulling the average up. Taking the logarithm compresses that long tail and gives a distribution that is much closer to symmetric, which suits linear models better and keeps a few luxury listings from dominating the error.

2.3 Why the problem matters

A working price model has a few practical uses. A new host with no pricing history could get a starting estimate. A guest could sanity-check whether a listing looks overpriced for its size and area. An analyst could study which features move the price the most across different cities. Our goal is not to build a production pricing engine, but to run a clean end-to-end data science project and to be honest about what the model can and cannot do.

2.4 Expected outcome

We expected location and capacity to be the strongest signals, and we expected a tree-based model to beat the linear ones because price does not scale in a simple straight line with the features. The results confirmed both expectations, which we discuss in Sections 8 and 9.

3 Dataset description

3.1 Source and format

The data comes from Kaggle as a single CSV file. It was originally released for a Deloitte machine learning competition. The file we used is `train.csv`. It contains 74,111 listings and 29 columns. The dataset is public and free to use for educational and research purposes under the terms shown on the Kaggle page.

3.2 Cities covered

The listings come from six US cities. The split is uneven, which matters later when we read city-level effects:

Table 1: Number of listings per city (full dataset).

City	Listings
New York City (NYC)	32,349
Los Angeles (LA)	22,453
San Francisco (SF)	6,434
Washington DC (DC)	5,688
Chicago	3,719
Boston	3,468

New York and Los Angeles together make up about three quarters of the rows. Boston and Chicago are the smallest groups.

3.3 Features

The 29 columns fall into a few groups. There are numeric features such as `accommodates`, `bathrooms`, `bedrooms`, `beds`, `number_of_reviews`, and `review_scores_rating`. There are categorical features such as `property_type`, `room_type`, `bed_type`, `cancellation_policy`, and `city`. A few columns are booleans stored as text, like `cleaning_fee`, `instant_bookable`, `host_has_profile_pic`, and `host_identity_verified`. The rest are free text or identifiers we did not use for modeling, including `description`, `name`, `amenities`, `thumbnail_url`, the review dates, and `host_response_rate`. Two columns, `latitude` and `longitude`, give the exact location of each listing.

3.4 Target and target range

The target `log_price` has a mean of about 4.78 and a median of about 4.71. Converting the median back to dollars gives a typical nightly price near \$111. The log values run from 0 up to about 7.6, which in dollars is roughly \$1 to \$2,000. The very low values look like data entry problems rather than real prices, and we come back to that in the limitations.

3.5 Limitations of the data

Three limitations stand out. First, the data is a snapshot from around 2017 and 2018, so the prices are out of date and should not be read as current market values. Second, the geography is narrow, only six large US cities, so nothing here transfers to smaller towns or to other countries.

Third, several columns have missing values, which we handle in the next section. The dataset has no duplicate rows and no duplicate listing IDs, which we checked before doing anything else.

4 Preprocessing and cleaning

We wrote the cleaning steps as a single function so that the training set and the test set go through exactly the same transformations. This is the part of the pipeline where data leakage usually creeps in, so we were careful to fit everything (the encoders and the scaler) on the training data only.

4.1 Train and test split done by us

The Kaggle file is a single table. We split it ourselves into 80 percent for training and 20 percent for testing, using a fixed `random_state` of 42 so the split is always the same. That gives 59,288 training rows and 14,823 test rows. The test set is put aside at the very start and is not touched again until the final evaluation in Section 8.

4.2 Dropping unused columns

We removed columns that a simple model cannot use well: identifiers (`id`), free text (`description`, `name`, `amenities`), the image link (`thumbnail_url`), and the date columns (`first_review`, `last_review`, `host_since`). We also dropped `zipcode`, `neighbourhood`, and `host_response_rate`. The first two overlap with the latitude and longitude we already keep, and `host_response_rate` is missing for a quarter of the rows and stored as a string with a percent sign.

4.3 Missing values

A few columns have missing entries. The worst offenders are `host_response_rate` (about 25 percent missing) and `review_scores_rating` (about 23 percent missing, because new listings have no reviews yet). The review dates are missing for about 21 percent of listings for the same reason. For the numeric features we keep (`bathrooms`, `bedrooms`, `beds`, `review_scores_rating`), we fill the gaps with the median of the training set. The median is safer than the mean here because these columns are skewed. For categorical columns we replace any missing value with the string `missing` so that it becomes its own category instead of breaking the encoder.

4.4 Encoding categorical variables

The categorical columns are text, and scikit-learn models need numbers. We used `LabelEncoder` to turn each category into an integer. The encoders are fitted on the training set. When we later apply them to the test set, any category the encoder has never seen is mapped to a safe default so the pipeline does not crash. Label encoding keeps the feature matrix compact, which matters given how many rows we have.

4.5 Scaling

We scaled the numeric features with `StandardScaler`, again fitting the scaler on the training data and reusing it on validation and test. Scaling helps the linear models and the Ridge penalty behave sensibly. The Random Forest does not need scaled inputs, but applying the same scaler to all three models keeps the code simple and does not hurt the tree.

5 Data visualization and exploratory data analysis

Before modeling we spent time looking at the data. The goal was to understand the shape of each variable, spot problems, and get a first idea of which features carry price information.

5.1 Distribution of the numeric features

Figure 1 shows histograms of the six main numeric columns. Most listings are small. Capacity is usually 2 to 4 guests, and one or two bedrooms is the norm. The `number_of_reviews` column has a very long tail, with a handful of listings collecting hundreds of reviews while most have only a few. The rating column is squeezed up near 90 to 100, so on its own it separates listings poorly.

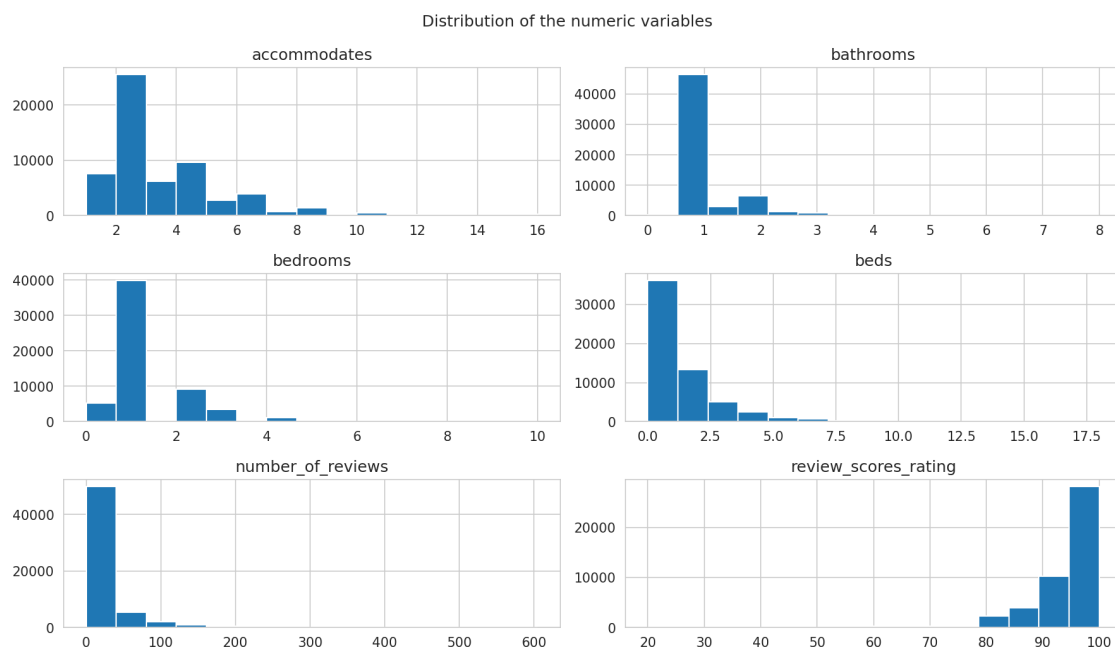


Figure 1: Distribution of the numeric variables in the training set. Capacity, bedrooms, bathrooms, and beds are right-skewed; ratings cluster near the top of the scale.

5.2 Distribution of the categorical features

Figures 2, 3, and 4 show the three categorical features that matter most. Apartments dominate the property types, houses are a distant second, and everything else is rare. For room type, entire homes and private rooms make up almost the whole dataset, while shared rooms are uncommon. The city plot repeats what the table in Section 3 already showed: NYC and LA carry most of the weight.

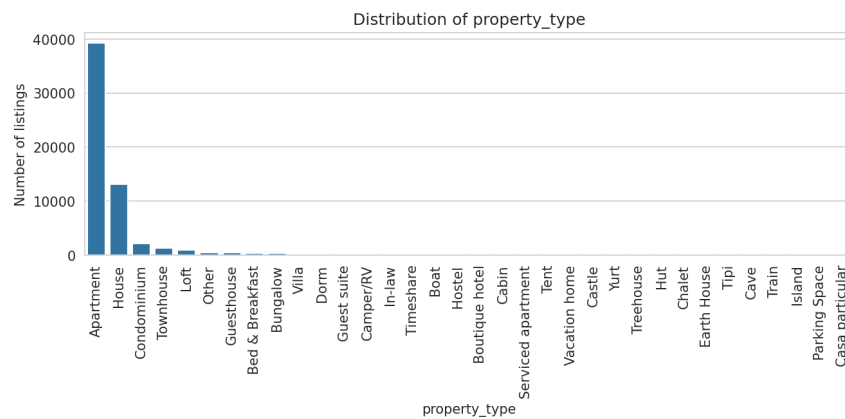


Figure 2: Distribution of property type. Apartments and houses account for the large majority of listings.

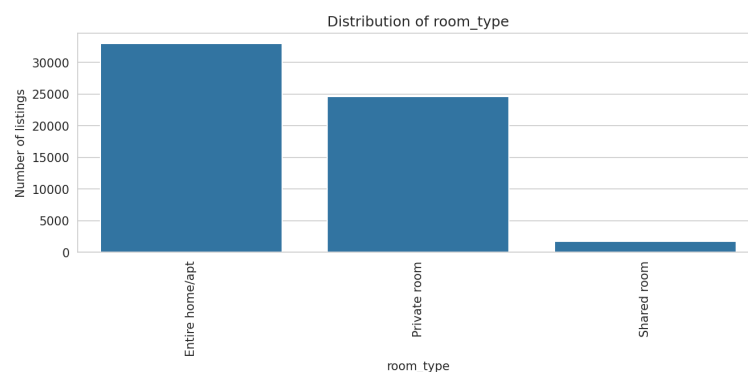


Figure 3: Distribution of room type. Shared rooms are rare, which is worth remembering when we read the price boxplots.



Figure 4: Number of listings per city. The class imbalance is strong, with NYC and LA far ahead of the rest.

5.3 The target variable

Figure 5 shows the distribution of `log_price`. After the log transform the shape is close to a bell curve, centered a little below 5 in log units. This is exactly why the log price is easier to model than the raw dollar price. There is a small spike at the very bottom of the range that corresponds to the suspicious near-zero prices we mentioned earlier.

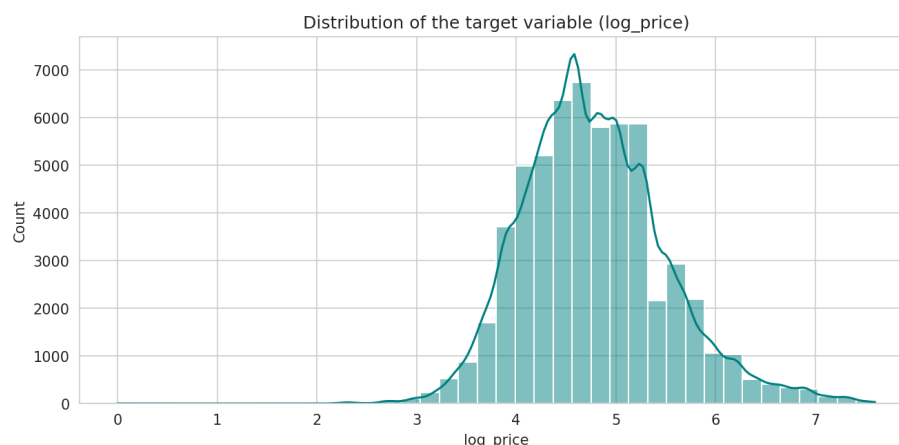


Figure 5: Distribution of the target variable `log_price`. The log transform gives a roughly symmetric distribution suitable for regression.

5.4 Price by city and by room type

Figure 6 splits the price by city and by room type. San Francisco and Boston have the highest median prices, and Chicago the lowest. The room type effect is clear and intuitive: entire homes cost more than private rooms, which cost more than shared rooms. These two plots already suggest that room type and location will be strong predictors.

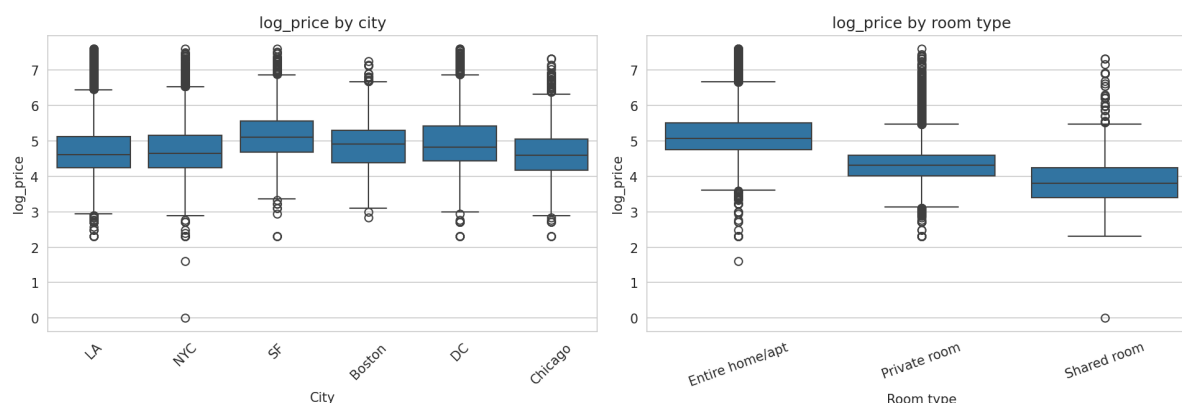


Figure 6: `log_price` by city (left) and by room type (right). Location and room type both shift the price in a visible and sensible way.

5.5 Correlation with the target

Figure 7 is the correlation matrix after encoding. Reading the `log_price` row, the strongest linear links are with `room_type` (about -0.61 , because entire homes get a low encoded value and shared rooms a high one) and `accommodates` (about $+0.57$). Then come `bedrooms` ($+0.48$), `beds` ($+0.44$), and `bathrooms` ($+0.35$). Latitude, longitude, and the number of reviews show almost no linear correlation on their own. That does not make them useless. A tree model can still pick up location through non-linear splits, which is exactly what happens in Section 8.

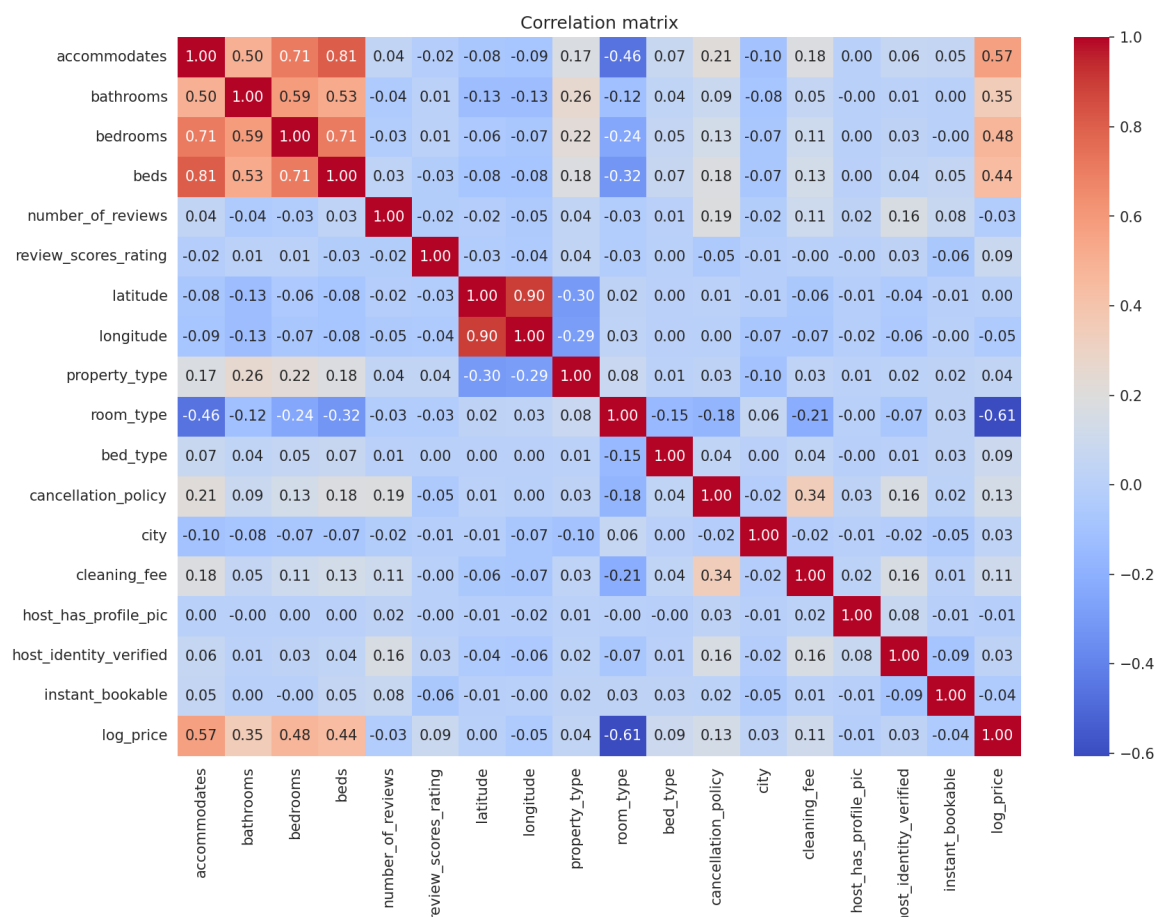


Figure 7: Correlation matrix of the encoded features and the target. Capacity-related features correlate positively with price; room type correlates strongly in the opposite direction because of its encoding.

6 Feature engineering

Our feature engineering is deliberate rather than heavy. For a first honest baseline we preferred a small, well-understood feature set over a large one full of noise.

6.1 Feature selection

From the 29 original columns we kept 17 as model inputs: 8 numeric features and 9 categorical features. We dropped the text, the identifiers, and the date columns for the reasons given in Section 4. Keeping the feature set small makes the models faster to train and much easier to interpret.

6.2 Encoding and scaling as engineered features

The main transformations are the label encoding of the categorical columns and the standard scaling of the numeric ones. Both are fitted on the training data only. Encoding turns text categories into integers the models can read. Scaling puts the numeric features on a common range so the linear models and the Ridge penalty treat them fairly.

6.3 Latitude and longitude as location proxies

We chose to keep the raw coordinates instead of the neighbourhood name. Coordinates are numeric, complete, and free of the spelling inconsistencies that plague text location fields. A single neighbourhood column would have needed one-hot encoding into hundreds of sparse columns. The correlation of the coordinates with price is weak on a straight line, but the Random Forest uses them heavily, which tells us they encode location value in a non-linear way.

6.4 What we chose not to do

We did not mine the `amenities` text or the `description` for extra signals. Both are rich but would need natural language processing to use properly, which is beyond the scope of this project. We list this as future work in Section 10.

7 Machine learning model training

The project asks for at least three suitable models. We trained three regression models and compared them on the same data with the same metrics.

7.1 Models

We used Linear Regression as a plain baseline, Ridge Regression as a regularized linear model, and a Random Forest Regressor as a non-linear tree-based model. Linear Regression fits a straight-line relationship between the features and the log price. Ridge does the same but adds a penalty on large coefficients to reduce overfitting. Random Forest builds many decision trees on random subsets of the data and averages them, which lets it model interactions and non-linear effects that the linear models cannot reach.

7.2 Training setup

Inside the training file we made a second 80/20 split into a training part and a validation part, again with `random_state` of 42. We fit each model on the scaled training part and measured it on the validation part. The Random Forest uses 200 trees with a fixed random seed. We left the other hyperparameters at their defaults for a fair and reproducible first comparison. The held-out test set from Section 4 stays untouched until the final evaluation.

7.3 Why these models

The two linear models give a clear, interpretable reference point and train almost instantly. The Random Forest is the natural next step when a straight line is too simple, and it also reports feature importances, which help us explain what the model learned. This trio covers the useful range from a simple transparent model to a stronger black-box one without going all the way to methods that would be hard to justify in a course project.

8 Evaluation metrics and results

8.1 Metrics

For a regression task the right metrics are the mean absolute error (MAE), the root mean squared error (RMSE), and the coefficient of determination (R^2). MAE is the average size of the error in log units and is easy to read. RMSE punishes large errors more heavily. R^2 says how much of the variance in the price the model explains, where 1.0 is perfect and 0.0 is no better than always guessing the mean.

8.2 Validation results

Table 2 shows the scores of the three models on the validation set. The two linear models land in almost exactly the same place. With standardized features the default Ridge penalty barely changes the coefficients, so regularization is not what limits the linear models here. The Random Forest is clearly ahead on all three metrics.

Table 2: Model comparison on the validation set. Lower MAE and RMSE are better; higher R^2 is better.

Model	MAE	RMSE	R^2
Random Forest	0.288	0.397	0.698
Ridge Regression	0.368	0.491	0.539
Linear Regression	0.368	0.491	0.539

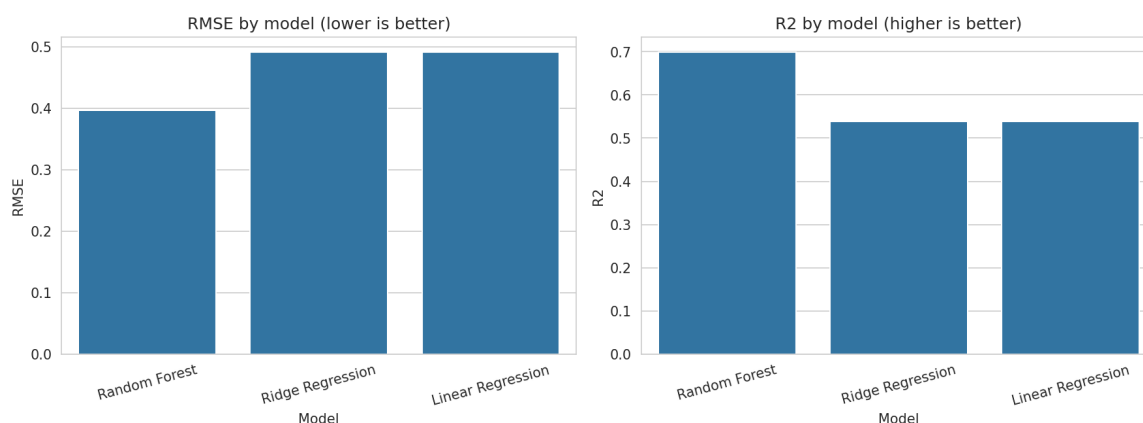


Figure 8: RMSE (left) and R^2 (right) for the three models on the validation set. The Random Forest wins on both.

8.3 Feature importance

Figure 9 shows what the Random Forest relied on. Room type is by far the most important feature, at about 37 percent of the total importance. Longitude and latitude come next, together around 28 percent, which confirms that location carries a lot of price information even though its linear correlation was weak. Bathrooms follow at about 12 percent. This ranking matches the story from the EDA: what kind of space you rent and where it is drive most of the price.

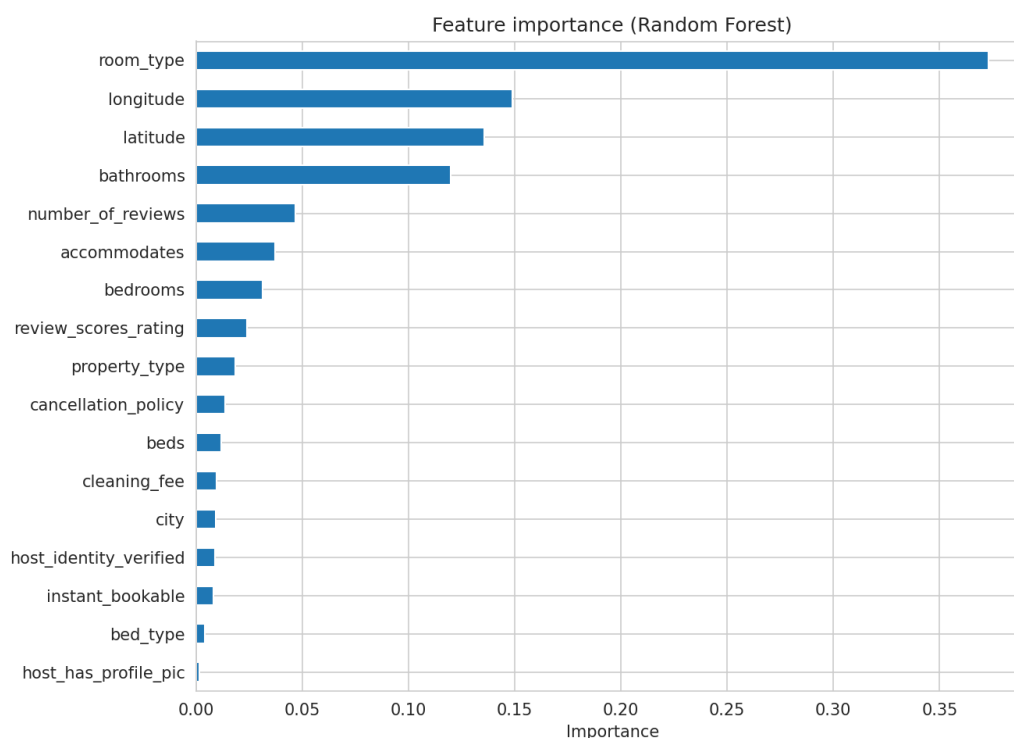


Figure 9: Feature importance from the Random Forest. Room type dominates, followed by the geographic coordinates and the number of bathrooms.

8.4 Final results on the held-out test set

We then ran the best model, the Random Forest, on the 20 percent test set that it had never seen. We reused the encoders and the scaler fitted on the training data, with no refitting, to avoid leakage. Table 3 shows the result.

Table 3: Final Random Forest results on the held-out test set.

Metric	MAE	RMSE	R^2
Random Forest (test)	0.287	0.399	0.691

The test scores almost match the validation scores. That is the sign we wanted: the model generalizes and is not overfitting. Figure 10 plots predictions against real values. Most points sit close to the diagonal, with more scatter at the two extremes where the data is thinner. Figure 11 compares the distribution of predicted prices with the real ones. The predicted distribution is a little narrower, which is normal because a model that minimizes squared error tends to pull extreme predictions toward the middle.



Figure 10: Random Forest predictions against real `log_price` on the validation set. The dashed line is the perfect-prediction diagonal.

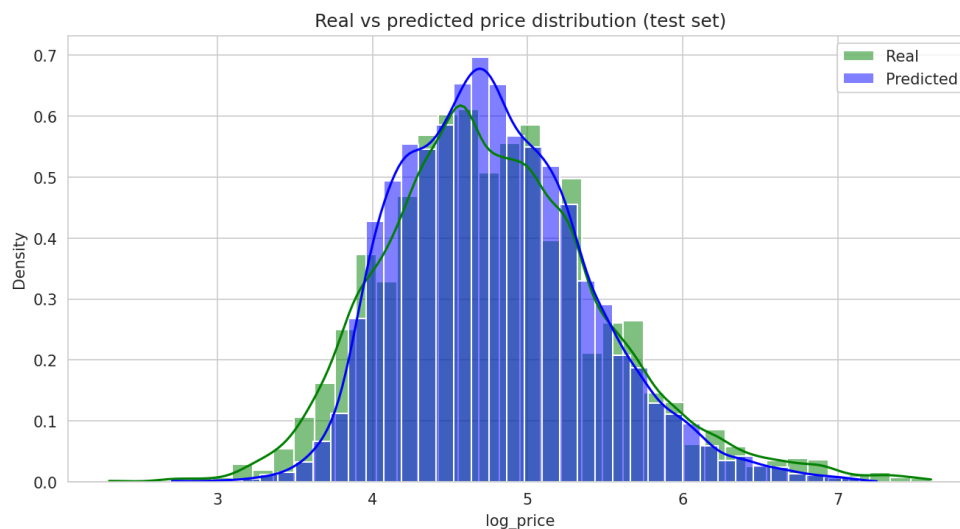


Figure 11: Real against predicted price distribution on the test set. The predictions cover the same range but are slightly less spread out than the real prices.

8.5 Honest reading of the results

An MAE of 0.287 in log units means a typical prediction is off by roughly 30 percent of the true nightly price once converted back to dollars. That is good enough to give a host a ballpark starting figure, but not precise enough to set a real price automatically. An R^2 near 0.69 means the model explains about two thirds of the variation in price, and leaves one third unexplained. A large part of that missing third is demand, seasonality, and listing quality that our features simply do not capture.

9 Responsible AI practices

Responsible use is part of the assignment, and it also matters for a topic like pricing that touches real money.

9.1 Privacy and personal data

We removed the host-related and free-text columns, so the model does not learn from anything that identifies a specific person. The features we kept describe the property and its location, not the host. The dataset is public and was released for a competition, but we still treated it as if it could contain sensitive traces and left those columns out.

9.2 Bias and fairness

Two kinds of bias are worth flagging. The first is geographic. The data is dominated by NYC and LA, so the model has seen far fewer examples from Boston or Chicago and will be less reliable there. The second is that a price model trained on past listings can copy and reinforce existing price gaps between neighbourhoods. If a tool like this were used to recommend prices, it could quietly push already-expensive areas higher and keep cheaper areas low. We are not deploying anything, but this is the sort of effect a real pricing system would need to watch.

9.3 Data leakage

We were careful to avoid leakage. The test set is separated before any cleaning decision is made. The medians used to fill missing values, the label encoders, and the scaler are all fitted on the training data only and then applied to the test data unchanged. No test information feeds back into preprocessing or model selection.

9.4 Honest reporting

We report MAE, RMSE, and R^2 on a test set the model never saw, not just the best score from training. We do not claim the model is accurate or finished. We state plainly that the errors are around 30 percent and that a third of the price variance is unexplained.

9.5 Real-world risks

The most obvious risk is someone treating the output as a real price today. The data is from 2017 and 2018, and prices have moved a lot since then, so any dollar figure here is out of date. A host who trusted a too-low estimate would lose income, and a guest who trusted a too-high one would overpay. The safe reading of this model is as a rough guide and a learning exercise, not a decision tool.

9.6 Use of AI tools

We used AI assistants for debugging, for explaining error messages, and for improving the grammar of this report. We reviewed and edited the text ourselves, and we can explain every step of the notebook and every decision in this report.

10 Conclusion and future work

10.1 Main findings

We built a full pipeline that predicts Airbnb log prices from property and location features. We split the Kaggle file ourselves into training and test sets, cleaned and encoded the data, explored it with statistics and plots, and compared three regression models. The Random Forest was the best by a clear margin, reaching an R^2 of about 0.69 on data it had never seen. The two linear models tied at about 0.54. The features that matter most are room type, location, and the number of bathrooms, which lines up with common sense about how Airbnb prices work.

10.2 What we learned

The project taught us how much of data science happens before the model. Cleaning, encoding, and keeping the test set honest took more thought than fitting the models did. We also saw a concrete case where a feature with almost no linear correlation, the coordinates, turned out to be one of the most useful inputs for a tree model. That gap between linear correlation and tree importance was a good lesson in not trusting a single number.

10.3 Future work

There is plenty of room to improve. The `amenities` and `description` text could be mined with NLP to add features about what each listing offers. The neighbourhood could be brought back with a smarter encoding such as target encoding instead of being dropped. The Random Forest hyperparameters could be tuned with cross-validation, and stronger models like gradient boosting are worth trying. Finally, joining the listings to demand or seasonality data would attack the part of the price variance our current features cannot reach.

11 Team contribution

All three members took part in the analysis and the writing. The table below shows the main focus of each person. The work was shared and reviewed together, so these are lead roles rather than strict boundaries.

Table 4: Contribution of each team member.

Member	Main contribution
Mathieu	Data loading and the train/test split, preprocessing function, missing-value handling, and the code-quality pass on the notebook.
Serghei	Exploratory data analysis and visualization, correlation study, feature selection, and the EDA sections of the report.
Samy	Model training and comparison, evaluation on the test set, feature importance, and the responsible AI and conclusion sections.

Coding was done together in shared sessions, and each member can explain the whole pipeline for the presentation and the Q&A.

References

- [1] Rudy Mizrahi. *AirBnB Listings in Major US Cities (Deloitte ML Competition)*. Kaggle. <https://www.kaggle.com/datasets/rudymizrahi/airbnb-listings-in-major-us-cities-deloitte->
- [2] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12:2825–2830, 2011. <https://scikit-learn.org>
- [3] The pandas development team. *pandas documentation*. <https://pandas.pydata.org>
- [4] M. Waskom. *seaborn: statistical data visualization*. <https://seaborn.pydata.org>
- [5] J. D. Hunter. *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3):90–95, 2007. <https://matplotlib.org>

Acknowledgements

We would like to thank our instructors, Ajan Aye Hninn Khine and Ajan Naveed Sultan, for their guidance throughout this course. Their lectures gave us the foundations we needed to carry out this project from start to finish, and their feedback helped us think more carefully about how we clean data, choose models, and report results honestly.

As exchange students from ESIEA, we also want to say how much we appreciated the welcome we received at KMUTT and the chance to learn data science with Python in such a supportive environment. Thank you for your time and your patience.